

JFN — SPEC: Skilltree via Telegram (Yoda) — atualização a quente

Para: Claude Code na VM `~/JFN` (Ubuntu 24.04, `venv ~/JFN/.venv`, sempre `PYTHONPATH=.`). **Objetivo:** permitir **ver e atualizar a skilltree do projeto todo pelo Telegram (Yoda)**, sem SSH e sem reiniciar o serviço. "Skilltree" = o registro de capacidades `capabilities.yaml` (fonte única que o Yoda lê para tool-calling). Companheiro de `JFN-DOCUMENTO-MESTRE-CONSOLIDADO.md` (esta feature encaixa na Onda 1 — Orquestração — e usa o `capabilities.yaml` do §7 daquele documento). **Data:** 2026-06-08.

Invariantes (respeitar)

- Fonte única:** `~/JFN/capabilities.yaml`. O Yoda só chama `id`s presentes nele — nunca inventa ferramenta.
- Reload fail-safe:** se o YAML novo for inválido, manter o anterior e reportar o erro. Nunca derrubar o roteador.
- Mutação só admin:** `/skills_reload`, `/skills_sync`, `/skills_validate` restritos a `TELEGRAM_ADMIN_IDS`. **Nunca** incluir esses comandos no allowlist do modo guest.
- Sem restart:** o roteador do Yoda lê as tool-specs de um local mutável (`app.bot_data["yoda_tools"]`); `reload/sync` trocam essa lista a quente.

Comandos

Comando	Faz	Permissão
<code>/skills [filtro]</code>	Skilltree agrupada por domínio (✅ PRONTO / 💎 ONDA N)	todos
<code>/skill <id></code>	Detalhe de uma skill (rota, args, quando usar, status)	todos
<code>/skills_reload</code>	Relê <code>capabilities.yaml</code> do disco + reaplica tools	admin
<code>/skills_sync</code>	<code>git pull --ff-only</code> no projeto todo + reload + reaplica tools	admin

/skills_validate	Schema + checa se toda rota PRONTO existe no server.py	admin
------------------	---	-------

Arquivos

- **criar** `compliance_agent/skilltree.py` (registro vivo) — código abaixo.
- **criar** `compliance_agent/notifications/skilltree_handlers.py` (handlers) — código abaixo.
- **editar** bootstrap do bot Yoda (wiring, 3 passos abaixo).
- **editar** handler do `/lista` (snippet abaixo).
- **editar** `~/JFN/capabilities.yaml` (acrescentar as 5 capacidades de domínio `sistema`, abaixo).
- **editar** `.env`: `TELEGRAM_ADMIN_IDS=<seu_id_telegram>`.
- **dep:** `pyyaml` (já no projeto). Nenhuma dependência nova.

Pressupostos a CONFIRMAR contra o código real (honestidade)

- Handlers escritos para **python-telegram-bot v20 (async)**. Se o Yoda usa outra lib (aiogram/wrapper Hermes), a lógica é idêntica, só muda a assinatura — adaptar.
- As tool-specs vivem em `app.bot_data["yoda_tools"]` (suposição). O essencial: o roteador deve ler a lista de um objeto mutável. Se o gateway guarda em outro lugar, apontar `_reaplicar_tools` para ele.

1) `compliance_agent/skilltree.py`

```
"""Skilltree = registro vivo de capacidades (capabilities.yaml).
```

```
Fonte única consumida pelo roteador do Yoda (tool-calling). Suporta reload a quente, validacao fail-safe, diff e sync via git — SEM restart do servico. Invariante: se o YAML novo for invalido, mantem o estado anterior (nunca quebra o roteador). Yoda so chama ids presentes aqui.
```

```
"""
```

```
from __future__ import annotations
```

```

import hashlib
import subprocess
import threading
import time
from dataclasses import dataclass, field
from pathlib import Path

import yaml

CAP_PATH = Path.home() / "JFN" / "capabilities.yaml"
REPO = Path.home() / "JFN"
SERVER = REPO / "compliance_agent" / "server.py"

@dataclass
class SkillTree:
    path: Path = CAP_PATH
    capacidades: dict = field(default_factory=dict) # id -> cap
    meta: dict = field(default_factory=dict)
    sha: str = ""
    carregado_em: float = 0.0
    _lock: threading.Lock = field(default_factory=threading.Lock, repr=False)

    # ----- parsing / validacao -----
    def _parse(self, raw: str) -> dict:
        doc = yaml.safe_load(raw)
        if not isinstance(doc, dict) or "capacidades" not in doc:
            raise ValueError("capabilities.yaml sem chave 'capacidades'")
        caps: dict = {}
        for c in doc["capacidades"]:
            cid = c["id"] # KeyError -> invalido
            if cid in caps:
                raise ValueError(f"id duplicado: {cid}")
            caps[cid] = c
        return {"meta": doc.get("meta", {}), "capacidades": caps}

    # ----- mutacoes (fail-safe) -----
    def reload(self) -> dict:
        """Recarrega do disco. Em erro, mantem o estado atual e propaga a execucao
        raw = self.path.read_text(encoding="utf-8")
        novo = self._parse(raw) # valida ANTES de trocar
        sha = hashlib.sha256(raw.encode()).hexdigest()[:12]
        with self._lock:
            antes = set(self.capacidades)
            self.meta = novo["meta"]
            self.capacidades = novo["capacidades"]
            self.sha = sha

```

```

        self.carregado_em = time.time()
    depois = set(self.capacidades)
    return {
        "sha": sha,
        "total": len(depois),
        "add": sorted(depois - antes),
        "rm": sorted(antes - depois),
    }

def sync(self) -> dict:
    """git pull --ff-only no projeto todo, depois reload."""
    pull = subprocess.run(
        ["git", "-C", str(REPO), "pull", "--ff-only"],
        capture_output=True, text=True, timeout=60,
    )
    if pull.returncode != 0:
        raise RuntimeError(pull.stderr.strip() or "git pull falhou")
    commit = subprocess.run(
        ["git", "-C", str(REPO), "rev-parse", "--short", "HEAD"],
        capture_output=True, text=True,
    ).stdout.strip()
    res = self.reload()
    res["commit"] = commit
    return res

# ----- validacao de contrato -----
def validate(self) -> list[str]:
    """Schema minimo + toda rota PRONTO existe no server.py."""
    problemas: list[str] = []
    server_src = SERVER.read_text(encoding="utf-8") if SERVER.exists() else ""
    for cid, c in self.capacidades.items():
        for campo in ("agente", "tipo", "status", "descricao"):
            if campo not in c:
                problemas.append(f"{cid}: falta '{campo}'")
            if c.get("status") == "PRONTO" and c.get("tipo") == "http":
                base = (c.get("rota", "")).split("{")[0].rstrip("/") # ignora pa
                if base and base not in server_src:
                    problemas.append(f"{cid}: rota {c.get('rota')} ausente no ser
    return problemas

# ----- saidas -----
def tool_specs(self) -> list[dict]:
    """Tool-specs (JSON schema) que o gateway do Yoda carrega p/ tool-calling
    Ignora capacidades ainda nao implementadas (status ONDA N)."""
    specs = []
    for cid, c in self.capacidades.items():
        if str(c.get("status", "")).startswith("ONDA"):

```

```

        continue
    specs.append({
        "name": cid,
        "description": f"{c['descricao']}. Quando usar: {c.get('quando_usar', '')}",
        "input_schema": {
            "type": "object",
            "properties": {
                k: {"type": "string", "description": str(v)}
                for k, v in (c.get("args") or {}).items()
            },
        },
    })
    return specs

def render(self, filtro: str = "") -> str:
    """Markdown da skilltree, agrupada por dominio (para /skills)."""
    por_dom: dict[str, list] = {}
    for cid, c in sorted(self.capacidades.items()):
        blob = f"{cid} {c.get('descricao', '')} {c.get('dominio', '')}".lower
        if filtro and filtro.lower() not in blob:
            continue
        por_dom.setdefault(c.get("dominio", "outros"), []).append((cid, c))
    if not por_dom:
        return f"Nenhuma skill casa com '{filtro}'."
    linhas = [
        f"🌳 *Skilltree* - v{self.meta.get('versao', '?')} · "
        f"`{self.sha}` · {len(self.capacidades)} skills"
    ]
    for dom in sorted(por_dom):
        linhas.append(f"\n*{dom.upper()}*")
        for cid, c in por_dom[dom]:
            st = c.get("status", "")
            tag = "✅" if st == "PRONTO" else f"💠{st}"
            linhas.append(f"{tag} `{cid}` - {c.get('descricao', '')}")
    return "\n".join(linhas)

def detalhe(self, cid: str) -> str:
    c = self.capacidades.get(cid)
    if not c:
        return f"Skill `{cid}` nao existe. Use /skills para listar."
    args = c.get("args") or {}
    args_txt = "\n".join(f"  • `{k}`: {v}" for k, v in args.items()) or "  (nada)"
    alvo = c.get("rota") or c.get("comando") or "?"
    return (
        f"🌳 *{cid}* ({c.get('status', '')})\n"
        f"dominio: {c.get('dominio', '?')} · agente: {c.get('agente', '?')}\n"
        f"{c.get('descricao', '')}\n"
    )

```

```

        f"*Quando usar:* {c.get('quando_usar', '-')}\\n"
        f"*Acesso:* `{c.get('tipo', '?')} {alvo}`\\n"
        f"*Args:*\\n{args_txt}\\n"
        f"*Retorno:* {c.get('retorno', '-')}\\n"
    )

```

```

# singleton vivo do processo (carrega na importacao)
SKILLTREE = SkillTree()
SKILLTREE.reload()

```

2) `compliance_agent/notifications/skilltree_handlers.py`

```

"""Handlers Telegram do Yoda para a skilltree.

```

```

Alvo: python-telegram-bot v20+ (async). ADAPTE o wiring (funcao `registrar` e
onde as tool-specs vivem) ao gateway real do Yoda – ver nota no fim.
Mutacoes (reload/sync/validate) sao restritas a admin (TELEGRAM_ADMIN_IDS no .env
NUNCA inclua estes comandos no allowlist do modo guest.

```

```

"""

```

```

from __future__ import annotations

```

```

import os

```

```

from telegram import Update

```

```

from telegram.constants import ParseMode

```

```

from telegram.ext import CommandHandler, ContextTypes

```

```

from compliance_agent.skilltree import SKILLTREE

```

```

ADMIN = {int(x) for x in os.getenv("TELEGRAM_ADMIN_IDS", "").split(",") if x.strip()}
MD = ParseMode.MARKDOWN

```

```

def _admin(update: Update) -> bool:

```

```

    return bool(update.effective_user and update.effective_user.id in ADMIN)

```

```

def _reaplicar_tools(ctx: ContextTypes.DEFAULT_TYPE) -> None:

```

```

    """Troca as tool-specs vivas que o roteador do Yoda usa no tool-calling.

```

```

    ADAPTE a chave/local conforme o gateway real (exemplo usa bot_data)."""

```

```

    ctx.application.bot_data["yoda_tools"] = SKILLTREE.tool_specs()

```

```

async def cmd_skills(update: Update, ctx: ContextTypes.DEFAULT_TYPE) -> None:
    filtro = " ".join(ctx.args) if ctx.args else ""
    await update.message.reply_text(SKILLTREE.render(filtro), parse_mode=MD)

async def cmd_skill(update: Update, ctx: ContextTypes.DEFAULT_TYPE) -> None:
    if not ctx.args:
        await update.message.reply_text("Uso: /skill <id>")
        return
    await update.message.reply_text(SKILLTREE.detatle(ctx.args[0]), parse_mode=MD)

async def cmd_skills_reload(update: Update, ctx: ContextTypes.DEFAULT_TYPE) -> No
    if not _admin(update):
        await update.message.reply_text("🚫 só admin.")
        return
    try:
        r = SKILLTREE.reload()
        _reaplicar_tools(ctx)
        msg = f"🔄 Skilltree recarregada · `{r['sha']}` · {r['total']} skills"
        if r["add"]:
            msg += f"\n➕ {'', '.join(r['add'])}"
        if r["rm"]:
            msg += f"\n➖ {'', '.join(r['rm'])}"
        await update.message.reply_text(msg, parse_mode=MD)
    except Exception as e: # noqa: BLE001
        await update.message.reply_text(f"❌ Falhou (skilltree antiga mantida): {

async def cmd_skills_sync(update: Update, ctx: ContextTypes.DEFAULT_TYPE) -> None
    if not _admin(update):
        await update.message.reply_text("🚫 só admin.")
        return
    await update.message.reply_text("⌚ git pull + reload...")
    try:
        r = SKILLTREE.sync()
        _reaplicar_tools(ctx)
        msg = (
            f"🔄 Projeto atualizado → commit `{r['commit']}`\n"
            f"Skilltree `{r['sha']}` · {r['total']} skills"
        )
        if r["add"]:
            msg += f"\n➕ {'', '.join(r['add'])}"
        if r["rm"]:
            msg += f"\n➖ {'', '.join(r['rm'])}"
        await update.message.reply_text(msg, parse_mode=MD)
    except Exception as e: # noqa: BLE001

```

```

        await update.message.reply_text(f"❌ Sync falhou: {e}")

async def cmd_skills_validate(update: Update, ctx: ContextTypes.DEFAULT_TYPE) ->
    if not _admin(update):
        await update.message.reply_text("🚫 só admin.")
        return
    probs = SKILLTREE.validate()
    if not probs:
        await update.message.reply_text("✅ Skilltree válida: schema ok e rotas F
    else:
        await update.message.reply_text("⚠ Problemas:\n• " + "\n• ".join(probs))

def registrar(app) -> None:
    """Chamar no bootstrap do bot, depois de construir o Application."""
    app.add_handler(CommandHandler("skills", cmd_skills))
    app.add_handler(CommandHandler("skill", cmd_skill))
    app.add_handler(CommandHandler("skills_reload", cmd_skills_reload))
    app.add_handler(CommandHandler("skills_sync", cmd_skills_sync))
    app.add_handler(CommandHandler("skills_validate", cmd_skills_validate))
    app.bot_data["yoda_tools"] = SKILLTREE.tool_specs() # estado inicial

# WIRING (adaptar ao Yoda real):
# 1) No bootstrap: from compliance_agent.notifications import skilltree_handlers
#                 skilltree_handlers.registrar(app)
# 2) No roteador do Yoda, ao montar a chamada de tool-calling, ler as tools de
#     app.bot_data["yoda_tools"] (em vez de uma lista estatica). Assim o reload/
#     sync troca as skills vivas sem reiniciar.
# 3) .env: TELEGRAM_ADMIN_IDS=<seu_id_telegram>

```

3) Wiring no bootstrap do Yoda

```

from compliance_agent.notifications import skilltree_handlers
# ... depois de construir o Application `app`:
skilltree_handlers.registrar(app)

```

No **roteador do Yoda**, ao montar a chamada de tool-calling, ler as ferramentas de um local mutável:

```

tools = ctx.application.bot_data.get("yoda_tools", []) # em vez de uma lista es
# ... passar `tools` para a API (tool-calling). reload/sync trocam essa lista a q

```


4) Incluir na `/lista` (ao final)

```
LISTA_SKILLTREE = (  
    "\n\n 🌳 *Skilltree (capacidades)*\n"  
    "/skills `[filtro]` - ver a árvore de skills\n"  
    "/skill `` - detalhe de uma skill\n"  
    "/skills\\_reload - recarregar do disco (admin)\n"  
    "/skills\\_sync - git pull + reload do projeto todo (admin)\n"  
    "/skills\\_validate - validar contrato (admin)"  
)  
# no fim da montagem do texto do /lista:  
texto += LISTA_SKILLTREE
```

5) Acrescentar ao `capabilities.yaml` (domínio `sistema`) — a skilltree se autodescreve

- {id: skills, agente: yoda, dominio: sistema, tipo: cli, comando: "telegram /s
descricao: "Lista a skilltree (capacidades) agrupada por dominio", quando_us
- {id: skill_detalhe, agente: yoda, dominio: sistema, tipo: cli, comando: "tele
descricao: "Detalhe de uma capacidade (rota, args, quando usar, status)", qu
- {id: skills_reload, agente: yoda, dominio: sistema, tipo: cli, comando: "tele
descricao: "Recarrega capabilities.yaml do disco e reaplica tools (fail-safe
- {id: skills_sync, agente: yoda, dominio: sistema, tipo: cli, comando: "telegr
descricao: "git pull --ff-only no projeto + reload + reaplica tools (admin)"
- {id: skills_validate, agente: yoda, dominio: sistema, tipo: cli, comando: "te
descricao: "Valida schema + checa que toda rota PRONTO existe no server.py (

6) Critérios de aceite

1. `/skills` retorna a árvore agrupada por domínio com status; `/skills massare` filtra.
2. `/skill anomalias` mostra rota, args, quando usar e status.
3. Editar o `capabilities.yaml` (adicionar uma capacidade), `/skills_reload` → resposta lista a nova em **+**, e ela passa a ser chamável pelo Yoda **sem restart**.
4. Corromper o YAML de propósito + `/skills_reload` → responde erro e **mantém** a skilltree anterior (Yoda segue funcionando).
5. `/skills_sync` faz `git pull` e reporta o `commit` curto + diff de skills.
6. `/skills_validate` lista capacidades PRONTO cuja rota não existe no `server.py` (ou

"válida").

7. Usuário não-admin recebe 🚫 só admin em reload/sync/validate.
8. /lista exibe a seção 🌳 Skilltree ao final.

7) Fluxo de operação

Editar capabilities.yaml (ou git push no repo) → /skills_validate → /skills_sync (ou /skills_reload se editou direto na VM) → /skills para conferir. Tudo a quente, com rollback automático em caso de YAML inválido.

8) Segurança

- TELEGRAM_ADMIN_IDS no .env (nunca no git). /skills_sync executa git pull --ff-only (sem merge sujo).
- Modo guest: expor no máximo /skills e /skill (read-only); jamais os comandos de mutação.